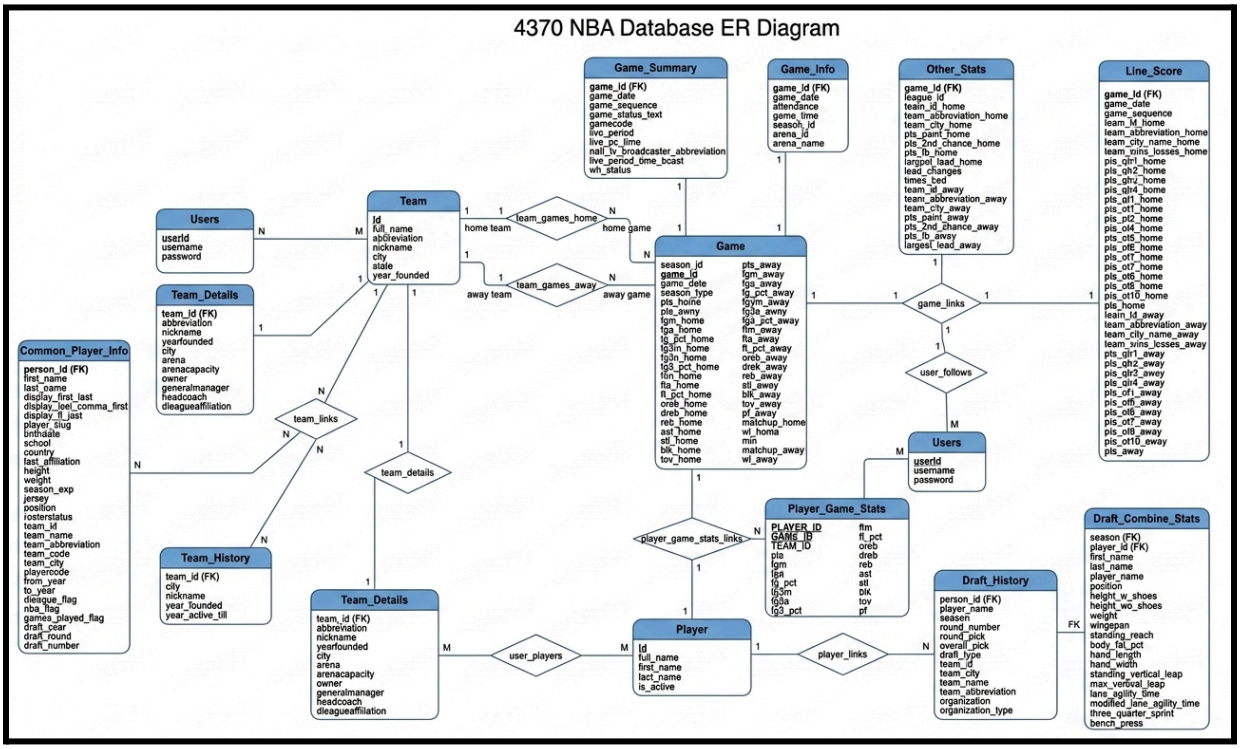
ER Model



ER to Table Conversion

User(
    id PK,
    username UNIQUE,
    password_hash,
    full_name
)

Team(
    id PK,
    full_name,
    abbreviation,
    nickname,
    city,
    state,
    year_founded
)

Game(
    game_id PK,

```
    season_id,
    game_date,
    season_type,
    team_id_home FK -> Team.id,
    team_abbreviation_home,
    team_name_home,
    wl_home,
    pts_home,
    plus_minus_home,
    team_id_away FK -> Team.id,
    team_abbreviation_away,
    team_name_away,
    wl_away,
    pts_away,
    plus_minus_away
)

Player(
    id PK,
    full_name,
    first_name,
    last_name,
    is_active
)

Player_Game_Stats(
    game_id PK, FK -> Game.game_id,
    player_id PK, FK -> Player.id,
    team_id FK -> Team.id,
    team_abbreviation,
    player_name,
    min,
    fgm,
    fga,
    fg_pct,
    reb,
    ast,
    stl,
    blk,
    pts,
```

plus_minus
)

Draft_History(
person_id PK, FK -> Player.id,
player_name,
season,
round_number,
round_pick,
overall_pick,
team_id FK -> Team.id,
team_city,
team_name,
team_abbreviation,
organization_type
)

Draft_Combine_Stats(
player_id PK, FK -> Player.id,
season,
player_name,
position,
height_wo_shoes,
weight,
wingspan,
standing_reach,
max_vertical_leap,
bench_press
)

Common_Player_Info(
person_id PK, FK -> Player.id,
display_first_last,
position,
school,
country,
height,
weight,
team_id FK -> Team.id,
team_name,

```
    team_abbreviation,
    draft_year,
    draft_round,
    draft_number
)

Team_Details(
    team_id PK, FK -> Team.id,
    abbreviation,
    nickname,
    city,
    arena,
    arena_capacity,
    owner,
    general_manager,
    headcoach
)

Team_History(
    team_id FK -> Team.id,
    city,
    nickname,
    year_founded,
    year_active_till
)

Game_Summary(
    game_id PK, FK -> Game.game_id,
    game_date_est,
    game_sequence,
    game_status_text,
    gamecode,
    home_team_id,
    visitor_team_id,
    season
)

Game_Info(
    game_id PK, FK -> Game.game_id,
    game_date,
```

attendance,
game_time
)

Other_Stats(
game_id PK, FK -> Game.game_id,
team_id_home,
pts_paint_home,
pts_2nd_chance_home,
pts_fb_home,
team_id_away,
pts_paint_away,
pts_2nd_chance_away,
pts_fb_away
)

Line_Score(
game_id PK, FK -> Game.game_id,
pts_qtr1_home,
pts_qtr2_home,
pts_qtr3_home,
pts_qtr4_home,
pts_home,
pts_qtr1_away,
pts_qtr2_away,
pts_qtr3_away,
pts_qtr4_away,
pts_away
)

Functional Dependencies Identified

id → username, password_hash, full_name
username → id, password_hash, full_name
team_id → team name, abbreviation, city, state, year_founded
game_id → season_id, game_date, season_type, home team, away team, home score, away score
player_id → full_name, first_name, last_name, is_active
(game_id, player_id) → team_id, minutes, points, rebounds, assists, steals, blocks, plus_minus
person_id → draft season, round_number, round_pick, overall_pick, draft team
player_id → combine measurements

team_id → arena, owner, general_manager, headcoach
game_id → game summary, game info, other stats, line score

Normalization

Example_Record(
  user_id, username, password_hash, full_name,
  game_id, season_id, game_date, season_type,
  home_team_id, home_team_name, home_team_abbreviation, pts_home,
  away_team_id, away_team_name, away_team_abbreviation, pts_away,
  player_id, player_name, player_pts, player_reb, player_ast,
  draft_season, round_number, round_pick, overall_pick,
  arena, owner, headcoach
)

This wide table contains update, insertion, and deletion anomalies. For example, team information would be repeated for every game, player information would be repeated for every stat row, and game information would be repeated for every player in that game.

1NF:
All attributes are atomic. Repeating game, team, player, and draft groups are separated.

2NF:
Attributes depending only on part of a composite key are removed. For example, player_name depends on player_id, not on the full key (game_id, player_id), so it belongs in Player.

BCNF Verification:
The original wide table is not in BCNF because it contains functional dependencies where the determinant is not a superkey.

For example:
team_id → team_name, team_abbreviation, city, state, year_founded
In the wide table, team_id does not determine all attributes such as game_id, player_id, player statistics, or draft information. Therefore, team_id is not a superkey, so this dependency violates BCNF. To fix this, team attributes are decomposed into the Team table.

Similarly:
player_id → full_name, first_name, last_name, is_active

player_id does not determine game statistics or draft data in the wide table, so player information is separated into Player.

Similarly:
game_id → season_id, game_date, season_type, home_team_id, away_team_id, scores
game_id does not determine every player's statistics or draft information, so game information is separated into Game and related game tables.

For Player_Game_Stats, the key is (game_id, player_id). The non-key attributes such as pts, reb, ast, stl, blk, and plus_minus depend on the full composite key. Since the determinant is the key, this relation satisfies BCNF.

After decomposition, each relation stores facts about one entity or relationship, and the major nontrivial functional dependencies have determinants that are primary keys or candidate keys. This reduces redundancy and avoids insertion, update, and deletion anomalies.

Final Normalized Relations and ER Comparison:
The tables produced through normalization are mostly the same as the tables from the ER model. Both approaches separate users, teams, games, players, player game statistics, draft information, team details, and game-related statistics into separate relations.

Some differences exist because the ER model keeps source-specific tables such as Game_Summary, Game_Info, Other_Stats, and Line_Score as separate tables. A strict BCNF decomposition could combine some of these because they are all determined by game_id. However, we decided to keep them separate because they represent different categories of game data and match the structure of the application data source. This improves organization and makes the database easier to query and maintain.

Therefore, we will use the ER model tables as the final normalized relations because they are already in BCNF that preserve the meaning of the original data.

The schema was implemented using the Docker-based MySQL instance used in class. The container runs MySQL 8.1.0 on port 33306. The database name is sports. The application connects using JDBC with username root and password mysqlpass. The schema is created by running ddl.sql, and demonstration data is loaded by running data.sql.